

Hardware Engineering Lab

FINAL PROJECT DOCUMENTATION

FPGA-Based Traffic-Light Controller with Pedestrian Request Button

TEAM	TEAM MEMBERS	STUDENT ID
A1	Nadia Afruz	1232739
	Md Riajul Islam	1230133
	Sayed Galib Hossen Rizvi	2200588
	Ronjon Sarker	2200002

Table of Contents

- Concept Description
- Project and Team Management
- Work Distribution
- Implemented and Integrated Technologies
- System Architecture and Operating Concept
- VHDL and FPGA Implementation
- Testbench Verification and Simulation Evidence
- Synthesis, Implementation, Timing, Utilization and Power Results
- PCB Design and Hardware Implementation
- Discussion, Problems Faced and Design Decisions
- Conclusion and Future Improvements
- Appendix A. VHDL Source-File Summary
- Appendix B. Nexys A7 Pin Mapping and XDC Constraints
- Appendix C. BOM Summary

1. Concept Description

In this Hardware Engineering Lab project, we developed an FPGA-based traffic-light controller for a simple two-direction road intersection. The controller was written in VHDL, tested using a simulation testbench and implemented in Vivado for the Nexys A7 FPGA board. A PCB interface design was also prepared to show how the external traffic-light LEDs and push buttons can be connected to the system.

The controller manages two road directions: North-South (NS) and East-West (EW). Each direction has red, yellow and green LED outputs. For pedestrians, the system uses WAIT and WALK indicators together with a push button for crossing requests. Since a real push button can create unstable or bouncing signals, the input is first synchronized and debounced before it is passed to the main controller.

The main control logic is based on a synchronous finite state machine. Each state represents one safe traffic-light condition. When the pedestrian button is pressed, the request is stored in the request_pending signal. The WALK signal is only enabled after both vehicle directions are red, so pedestrians and moving vehicles are never given permission at the same time.

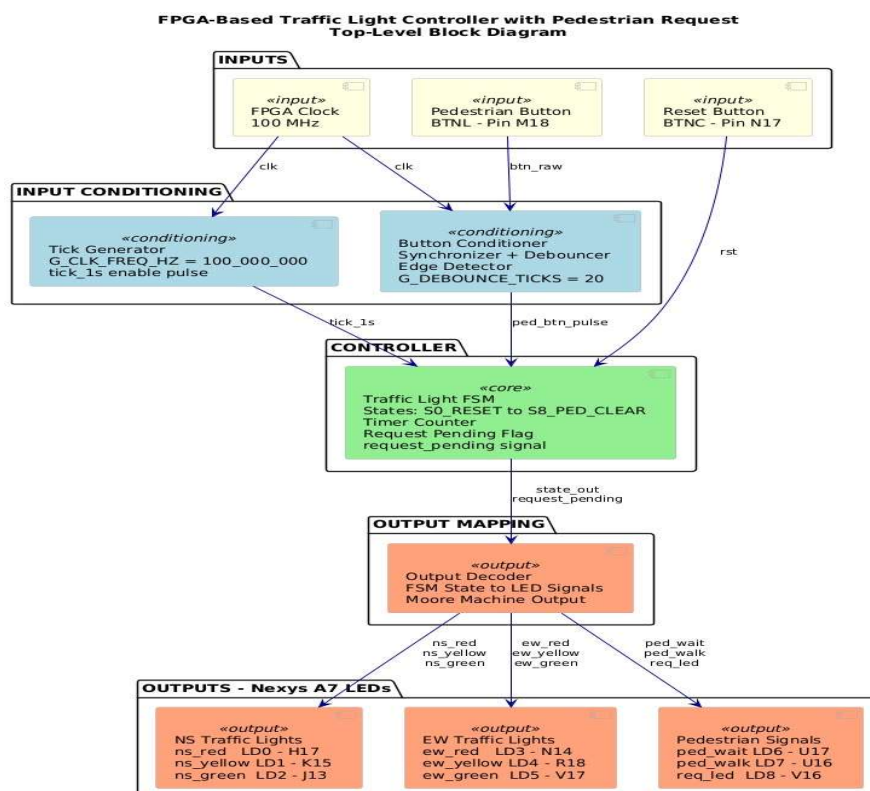


Figure 1: Top-level block diagram showing the inputs, input conditioning, controller core, output decoder and Nexys A7 LED assignments.

1.1 Project Objectives

The main objectives of this project are:

- Design a safe and easy-to-follow traffic-light control sequence for a two-direction intersection.
- Use a finite state machine so that every light phase follows a clear and predictable order.
- Implement the complete controller in synthesizable VHDL for FPGA-based hardware design.
- Add a pedestrian request button and serve the request only when both vehicle directions are in a safe red-light condition.
- Synchronize and debounce the pedestrian button so the controller receives one clean request signal.
- Generate a slow timing tick from the 100 MHz FPGA clock so that the traffic-light changes can be observed clearly.
- Verify reset behavior, the normal traffic sequence, pedestrian request handling and safety conditions using a VHDL testbench.
- Synthesize and implement the design in Vivado and generate the bitstream for the Nexys A7 FPGA board.
- Prepare the supporting PCB interface, including schematic, PCB layout, 3D view, BOM and Gerber output files.
- Document the development process, module structure, verification results, implementation results and team contribution.

1.2 Functional Requirements

ID	Requirement	Acceptance Criterion
FR-01	The NS direction shall provide RED, YELLOW and GREEN outputs.	Only one NS vehicle color shall be active during each normal vehicle phase.
FR-02	The EW direction shall provide RED, YELLOW and GREEN outputs.	Only one EW vehicle color shall be active during each normal vehicle phase.
FR-03	NS GREEN and EW GREEN shall never be active at the same time.	The testbench shall check this condition using a safety assertion.
FR-04	The controller shall include one pedestrian request button.	A debounced button press shall set the stored request_pending flag.
FR-05	The pedestrian request shall be served only during a safe all-red vehicle condition.	Pedestrian WALK shall be active only when NS and EW are both RED.

FR-06	The controller shall include a reset input.	Reset shall return the FSM to S0_RESET, an all-red start-up state.
FR-07	The design shall use configurable timing constants.	Phase timing can be changed in the VHDL package without rewriting the FSM.
FR-08	The design shall be synthesizable and implementable on the Nexys A7 board.	Vivado synthesis, implementation and bitstream generation shall complete successfully.

2. Project and Team Management

The project was divided into two main parts: the FPGA/VHDL design and the PCB interface design. For the digital part, we first defined the requirements and module structure, then wrote the VHDL code, simulated it with a testbench and completed synthesis, implementation and bitstream generation in Vivado. For the PCB part, the work moved from schematic design and component selection to layout, design checks, BOM, 3D view and Gerber output files.

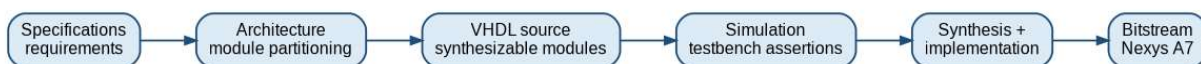


Figure 2: Digital design workflow used for the VHDL and FPGA implementation.



Figure 3: PCB design workflow used for the traffic-light interface board.

2.1 Work Distribution

The tasks were shared so that each team member had a main technical responsibility, while the whole team still supported integration, checking and documentation.

Team member	Main responsibility	Contribution to this project
Nadia Afruz	VHDL controller and FSM implementation	Implemented and supported the traffic-light FSM sequence, timing constants, request_pending behavior and VHDL integration in Vivado.
Md Riajul Islam	FPGA board setup, pin mapping and documentation	Worked on Nexys A7 board understanding, LED/button pin mapping, XDC connection explanation and documentation.
Sayed Galib Hossen Rizvi	Simulation and Vivado verification	Prepared and checked the VHDL testbench flow, waveform evidence, safety assertions, synthesis and implementation reports, timing/utilization/power results and bitstream evidence.
Ronjon Sarker	PCB design and external hardware interface	Worked on the Altium schematic, PCB layout and routing, component selection, 3D view, BOM and Gerber/output package for the traffic-light interface board.

Although each team member had a main responsibility, the project was completed through group collaboration. The documentation, final checking and project submission preparation were completed together by all team members.

3. Implemented and Integrated Technologies

3.1 VHDL

VHDL was used to describe the traffic-light controller as digital hardware. The design uses IEEE.STD_LOGIC_1164 for logic signals and IEEE.NUMERIC_STD for counters and numerical operations.

3.2 FPGA and Nexys A7

The FPGA is the target hardware for the VHDL design. The Vivado project targets the Artix-7 device xc7a100tcs324-1 on the Digilent Nexys A7-100T board. The board provides the 100 MHz clock, push buttons and LEDs used to test the prototype.

3.3 Vivado 2025.2

Vivado 2025.2 was used for the FPGA development flow, including simulation, synthesis, implementation, timing analysis, utilization and power reports, and bitstream generation.

3.4 Altium Designer

Altium Designer was used for the PCB part of the project. The PCB package includes schematic and PCB design files, Gerber outputs, a PDF view, a 3D view and a BOM spreadsheet.

3.5 GitHub

GitHub was used to keep the project files organized and traceable, including VHDL files, constraints, PCB files, documentation and generated reports.

4. System Architecture and Operating Concept

The top-level VHDL design connects four main blocks: the tick generator, the button conditioner, the traffic-light FSM and the output decoder. The tick_1s signal controls the duration of each phase. The s_btn_pulse signal gives the FSM a clean one-clock pedestrian request. The FSM stores the current state and request status, and the output decoder converts those states into LED outputs.

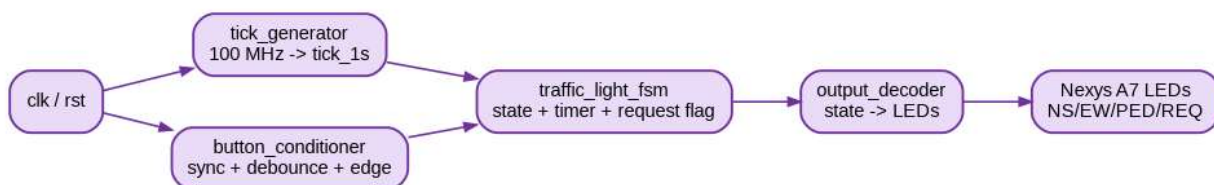


Figure 4: Implemented VHDL module structure based on the project source files.

4.1 Traffic-light state machine

The controller uses nine states. S0_RESET is the start-up state with both vehicle directions red. States S1 to S6 form the normal North-South and East-West traffic sequence, including yellow and all-red clearance phases. If a pedestrian request is pending at S6_ALL_RED_2, the FSM moves to S7_PED_WALK. After the walking time, S8_PED_CLEAR returns the crossing to WAIT before the vehicle sequence starts again.

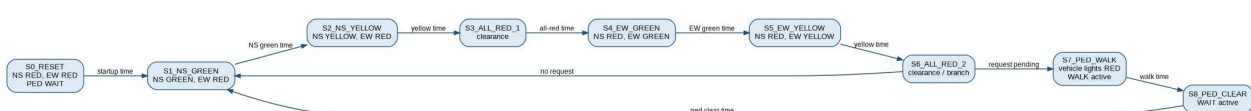


Figure 5: FSM sequence implemented in traffic_light_fsm.vhd.

State	Vehicle outputs	Pedestrian outputs	Transition condition
S0_RESET	NS RED, EW RED	WAIT active	Startup timer expires
S1_NS_GREEN	NS GREEN, EW RED	WAIT active	NS green timer expires
S2_NS_YELLOW	NS YELLOW, EW RED	WAIT active	Yellow timer expires
S3_ALL_RED_1	NS RED, EW RED	WAIT active	All-red clearance timer expires
S4_EW_GREEN	NS RED, EW GREEN	WAIT active	EW green timer expires
S5_EW_YELLOW	NS RED, EW YELLOW	WAIT active	Yellow timer expires
S6_ALL_RED_2	NS RED, EW RED	WAIT active	If request_pending = 1, go to S7; otherwise return to S1
S7_PED_WALK	NS RED, EW RED	WALK active	Walk timer expires and request flag is cleared
S8_PED_CLEAR	NS RED, EW RED	WAIT active	Pedestrian-clearance timer expires

4.2 Timing Constants

Constant	Value	Description
C_NS_GREEN_TIME	5	NS green duration used by the controller package.
C_EW_GREEN_TIME	5	EW green duration used by the controller package.
C_YELLOW_TIME	2	Yellow-light duration used after each green phase.
C_ALLRED_TIME	1	All-red clearance duration between vehicle phases.
C_WALK_TIME	4	Pedestrian WALK duration after a stored request is served.
C_PEDCLEAR_TIME	2	Pedestrian clearance duration before returning to vehicle traffic.
C_STARTUP_TIME	1	Initial all-red start-up duration after reset.

5. VHDL and FPGA Implementation

The controller was separated into small VHDL modules so that each part of the design could be understood, tested and reused more easily. The top-level file connects these modules using internal signals.

File	Entity / package	Main responsibility
traffic_light_pkg.vhd	traffic_light_pkg	Defines the FSM state type and phase timing constants.
tick_generator.vhd	tick_generator	Counts the 100 MHz clock and produces a one-cycle tick_1s enable pulse.
button_conditioner.vhd	button_conditioner	Synchronizes the raw pedestrian button, debounces it and generates btn_pulse.
traffic_light_fsm.vhd	traffic_light_fsm	Stores the current state, timer count and request_pending flag, and controls FSM transitions.
traffic_light_output_decoder.vhd	traffic_light_output_decoder	Maps FSM states to NS, EW, pedestrian and request-status LED outputs.
traffic_light_top.vhd	traffic_light_top	Connects all submodules for the FPGA implementation.
traffic_light_fsm_tb.vhd	traffic_light_fsm_tb	Provides the behavioral simulation testbench with reset, button stimulus and safety assertions.

5.1 Tick generator

The tick generator creates a slow enable pulse from the 100 MHz board clock. This keeps the complete design synchronous while making the traffic-light changes visible during operation.

```
entity tick_generator is
  generic (G_CLK_FREQ_HZ : integer := 100_000_000);
  port (clk : in std_logic; rst : in std_logic; tick_1s : out std_logic);
end entity tick_generator;
```

5.2 Button conditioner

The button conditioner prepares the pedestrian button signal before it reaches the FSM. It uses two flip-flops for synchronization, a debounce counter to filter bouncing and edge detection to produce one clean request pulse.

```
entity button_conditioner is
  generic (G_DEBOUNCE_TICKS : integer := 20);
  port (clk : in std_logic; rst : in std_logic;
        btn_raw : in std_logic; btn_pulse : out std_logic);
end entity button_conditioner;
```

5.3 FSM request policy

When `ped_btn_pulse` becomes active, the FSM stores the request in `r_req_pending`. The request does not immediately interrupt an active vehicle phase. Instead, it waits until `S6_ALL_RED_2`, where both roads are red, and then moves to `S7_PED_WALK`. The request flag is cleared during the pedestrian walk state.

```
if ped_btn_pulse = '1' then
  r_req_pending <= '1';
end if;
...
when S6_ALL_RED_2 =>
  if r_timer >= C_ALLRED_TIME-1 then
    if r_req_pending = '1' then
      r_state <= S7_PED_WALK;
    else
      r_state <= S1_NS_GREEN;
    end if;
  end if;
```

```

end if;
when S7_PED_WALK =>
  r_req_pending <= '0';

```

5.4 Output decoder

The output decoder starts from a safe default condition: both vehicle directions red and pedestrian WAIT active. It then changes only the signals needed for the current state. This makes the all-red condition the fallback behavior for reset and unused states.

State group	Decoder behavior
Default / all-red states	NS RED = 1, EW RED = 1 and PED WAIT = 1; all green/yellow outputs are off.
S1_NS_GREEN	NS GREEN is active; EW remains RED and pedestrian WAIT stays active.
S2_NS_YELLOW	NS YELLOW is active; EW remains RED and pedestrian WAIT stays active.
S4_EW_GREEN	EW GREEN is active; NS remains RED and pedestrian WAIT stays active.
S5_EW_YELLOW	EW YELLOW is active; NS remains RED and pedestrian WAIT stays active.
S7_PED_WALK	Both vehicle directions remain RED; pedestrian WALK is active and WAIT is off.

5.5 Nexys A7 LED and Button Mapping

Signal	Function	Nexys A7 component / pin from project sketch
clk	100 MHz FPGA clock	Board clock, 100 MHz
btn_ped	Pedestrian request button	BTNL - M18
rst	Reset button	BTNC - N17
ns_red	North-South red	LD0 - H17

ns_yellow	North-South yellow	LD1 - K15
ns_green	North-South green	LD2 - J13
ew_red	East-West red	LD3 - N14
ew_yellow	East-West yellow	LD4 - R18
ew_green	East-West green	LD5 - V17
ped_wait	Pedestrian WAIT	LD6 - U17
ped_walk	Pedestrian WALK	LD7 - U16
req_led	Stored request indicator	LD8 - V16

6. Testbench Verification and Simulation Evidence

The testbench checks the main behavior of the controller. It applies reset, runs the normal traffic sequence, sends a pedestrian button request and continues the simulation so the request handling can be observed in the waveform. It also includes safety assertions to detect unsafe cases, such as both vehicle greens being active together or pedestrian WALK being active with a vehicle green signal.

Test ID	Scenario	Expected result / evidence
TB-01	Reset is asserted for 100 ns.	The controller starts in S0_RESET with NS RED, EW RED and pedestrian WAIT active.
TB-02	The normal traffic cycle is allowed to run.	NS and EW phases alternate with yellow and all-red intervals.
TB-03	The pedestrian button is pressed.	The request is captured by the conditioner/FSM and req_led stays active until the pedestrian phase begins.
TB-04	The pedestrian request is served.	Pedestrian WALK becomes active only after both vehicle directions are red.
TB-05	Safety assertions run continuously.	The assertions check for simultaneous vehicle greens and pedestrian/vehicle conflicts.

6.1 Testbench Safety Assertions

```

assert NOT(tb_ns_grn='1' AND tb_ew_grn='1')
    report "SAFETY FAIL: NS+EW green together!" severity FAILURE;

assert NOT(tb_ped_walk='1' AND (tb_ns_grn='1' OR tb_ew_grn='1'))
    report "SAFETY FAIL: PED_WALK with vehicle green!" severity FAILURE;

```

6.2 Vivado Simulation Screenshots

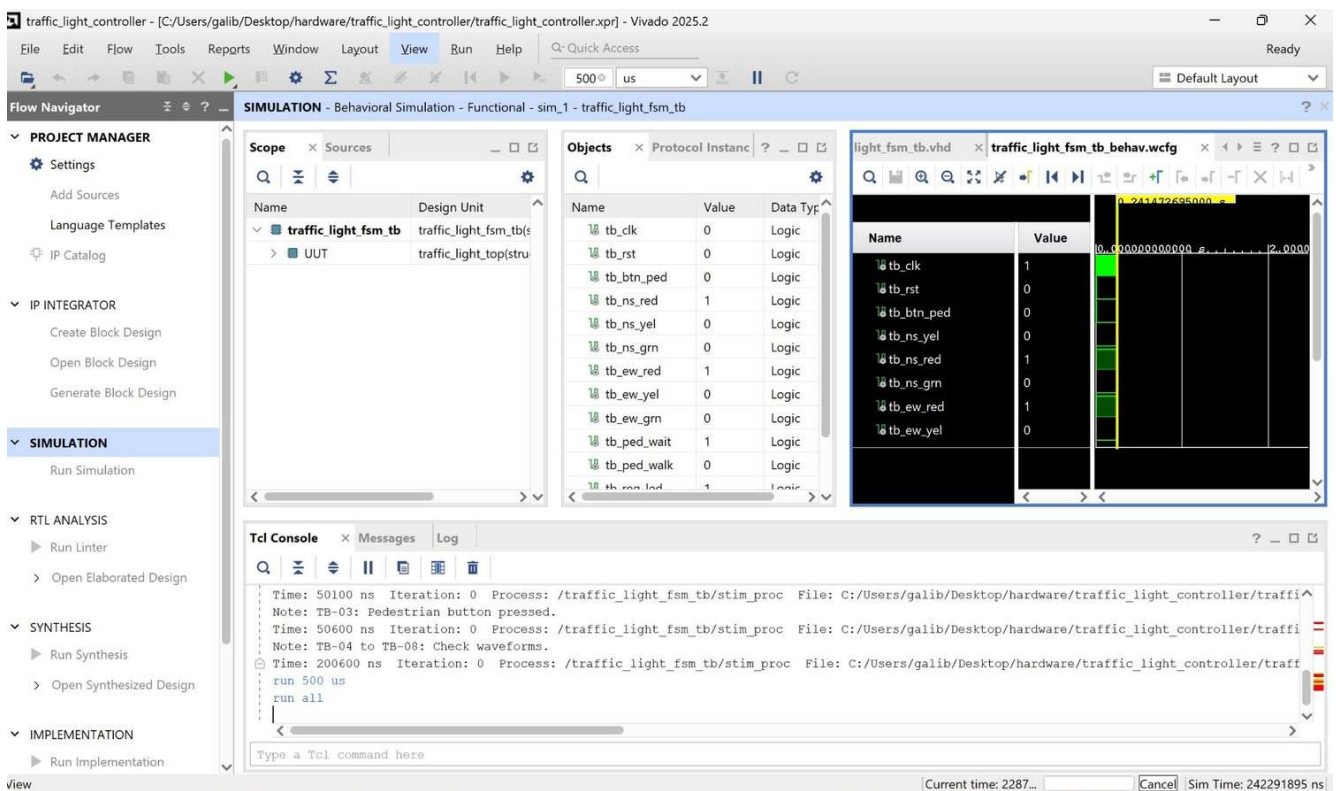


Figure 6: Behavioral simulation screenshot showing the reset/all-red output condition and the active waveform window.

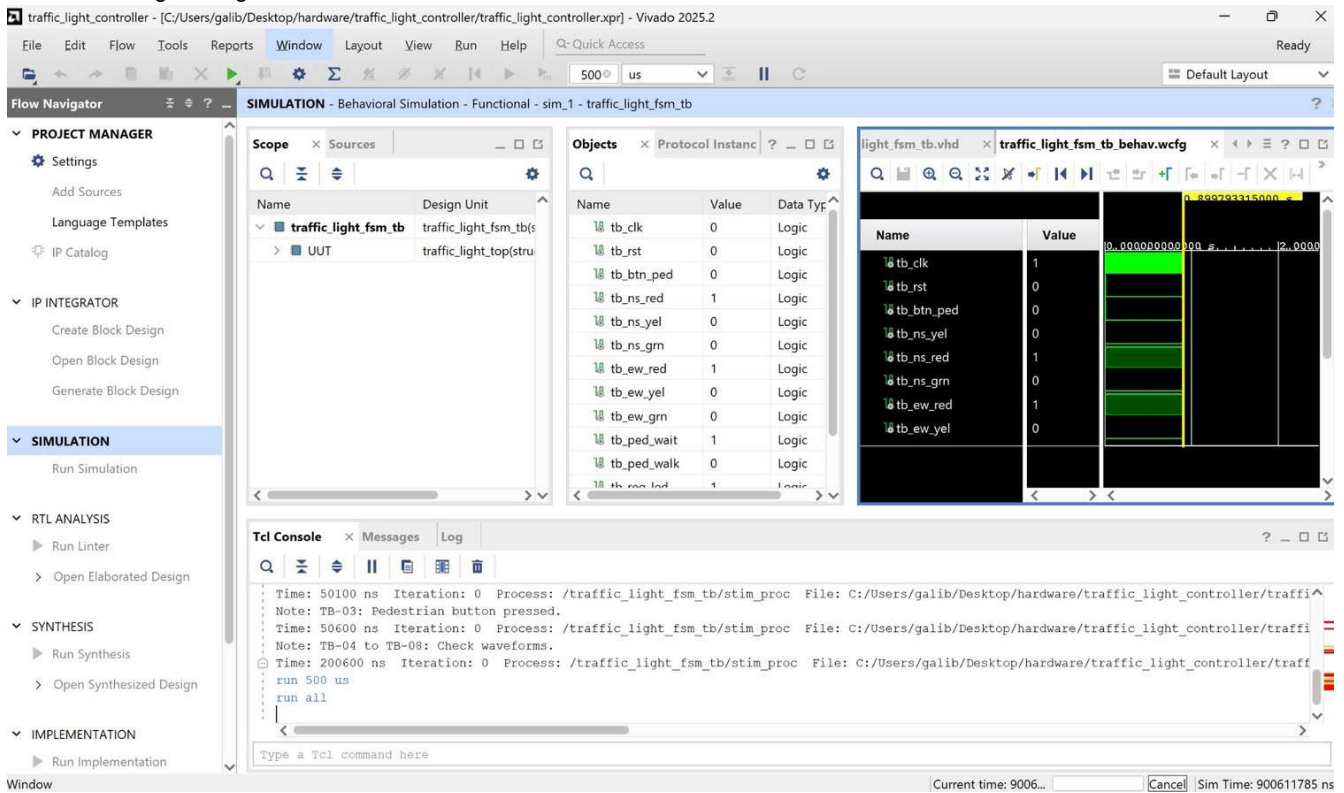


Figure 7: Behavioral simulation screenshot showing the output signal progression during the normal traffic-light sequence.

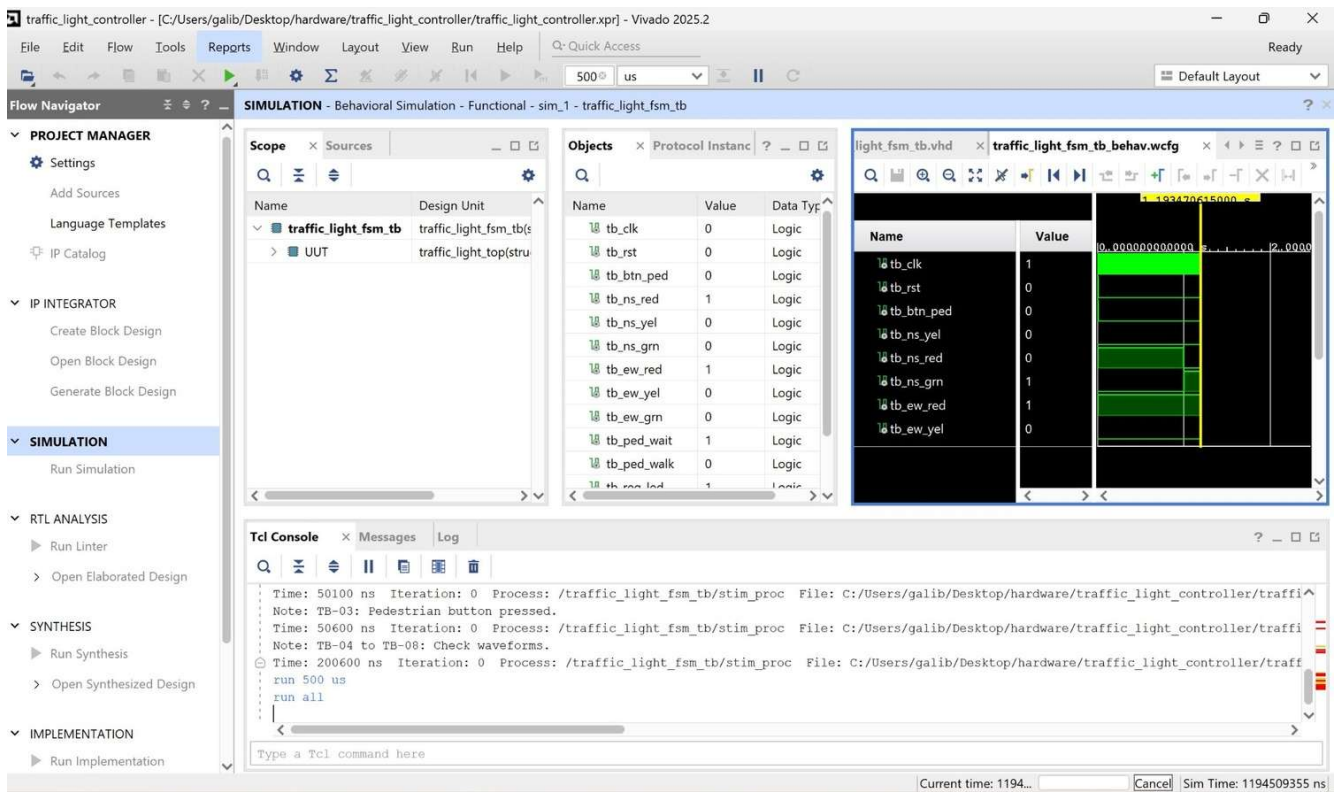


Figure 8: Behavioral simulation screenshot after the pedestrian button request is applied by the testbench.

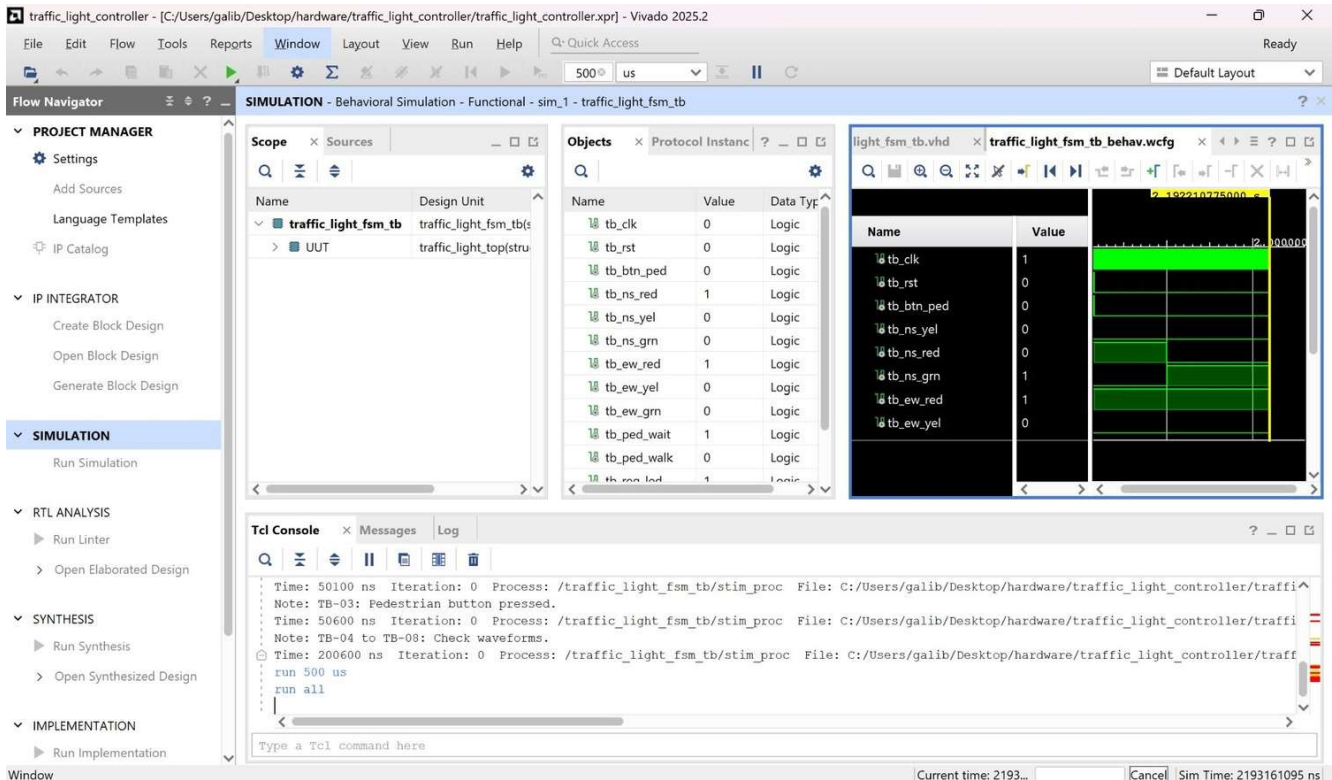


Figure 9: Later waveform screenshot showing that the controller continues the state/output sequence after the pedestrian request.

7. Synthesis, Implementation, Timing, Utilization and Power Results

The Vivado design was synthesized and implemented for the Artix-7 device on the Nexys A7-100T board. The generated reports show that the controller uses only a small part of the available FPGA resources and that the routed design satisfies the 100 MHz clock constraint.

Item	Result
Vivado version	Vivado v2025.2, 64-bit
Top-level design	traffic_light_top
Project target part	xc7a100tcsq324-1
Board part	digilentinc.com:nexys-a7-100t:part0:1.2
Design state for utilization/power/timing reports	Routed
Clock constraint	10.000 ns period, 100.000 MHz, waveform {0.000 5.000}
Bitstream status	Vivado GUI shows write_bitstream Complete

7.1 Timing Result

Metric	Value	Interpretation
Worst Negative Slack (WNS)	4.144 ns	Positive setup slack; no setup timing failure.
Total Negative Slack (TNS)	0.000 ns	No accumulated setup timing violation.
Setup failing endpoints	0	No setup endpoints failed.
Worst Hold Slack (WHS)	0.132 ns	Positive hold slack; no hold timing failure.
Total Hold Slack (THS)	0.000 ns	No accumulated hold timing violation.
Hold failing endpoints	0	No hold endpoints failed.
Worst Pulse Width Slack (WPWS)	4.500 ns	Positive pulse-width slack.
Pulse-width failing endpoints	0	No pulse-width endpoints failed.

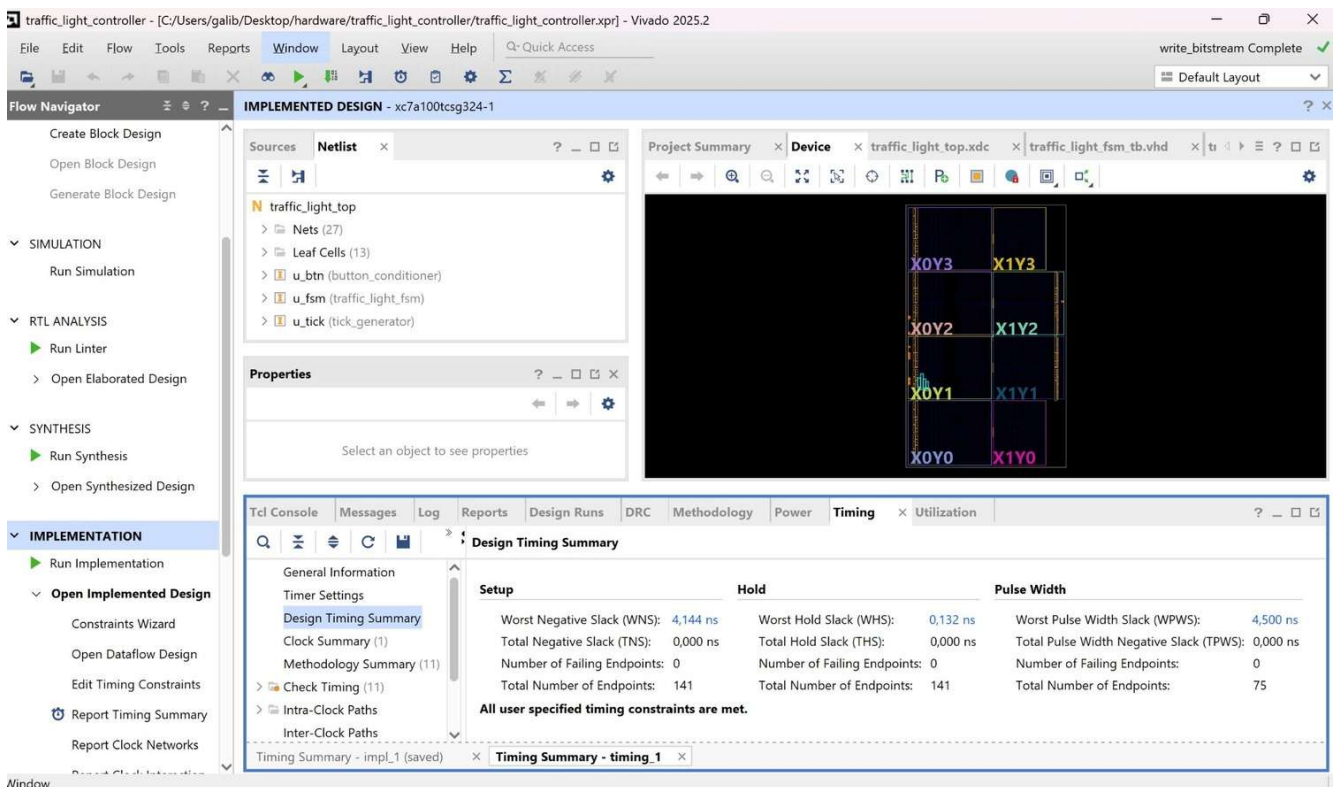


Figure 10: Vivado timing summary showing positive WNS, WHS and WPWS values with zero failing endpoints.

7.2 Utilization Result

Resource	Used	Available	Utilization (%)
Slice LUTs	110	63400	0.17%
Slice Registers	74	126800	0.06%
Slices	110	63400	0.17%
Bonded IOB	12	210	5.71%
BUFGCTRL	1	32	3.13%
DSP	0	240	0.00%
Block RAM Tile	0	135	0.00%

Implementation insight	Explanation
Low utilization	The routed report shows only 110 LUTs and 74 flip-flops. This confirms that the controller is small and suitable for the Nexys A7-100T.
No BRAM/DSP usage	The design is simple control logic, so it does not need block RAM or DSP resources.
IO usage	The 12 bonded I/O ports are used for clock/reset/button inputs and LED outputs.

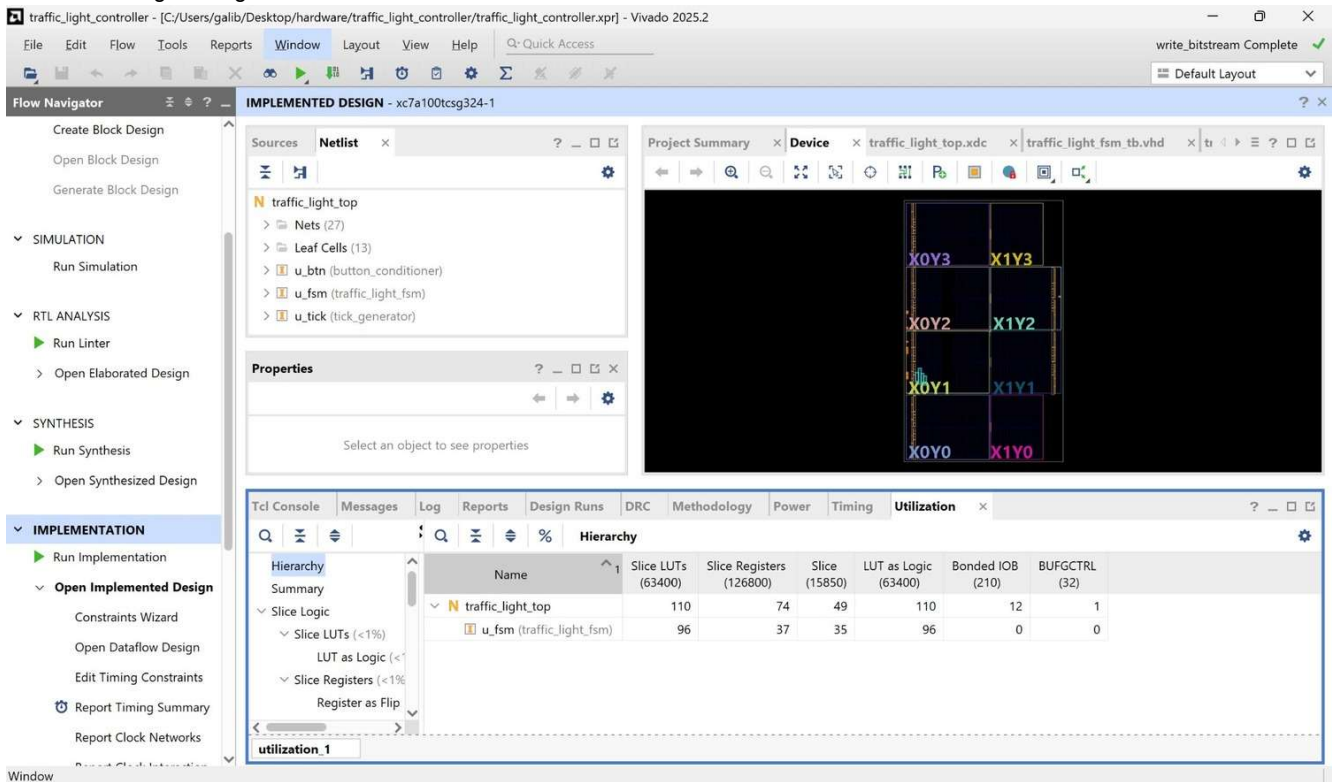


Figure 11: Vivado utilization screenshot showing low LUT, register and I/O usage.

7.3 Power Result

Power metric	Value	Comment
Total on-chip power	0.098 W	Estimated routed-design on-chip power.
Dynamic power	0.001 W	Very low because the design is mainly simple control logic.
Device static power	0.097 W	Main part of the estimated total power.
Junction temperature	25.4 °C	Estimated junction temperature for the report settings.
Maximum ambient	84.6 °C	Estimated maximum ambient temperature from the report.
Confidence level	Medium	The activity estimation is not fully simulation-annotated.

7.4 Bitstream Evidence

The Vivado screenshot shows that write_bitstream completed successfully. This confirms that the implementation flow reached bitstream generation. Since the hardware manager is not connected in the

screenshot, this image is used as evidence of bitstream generation, not as proof that the FPGA board was programmed.

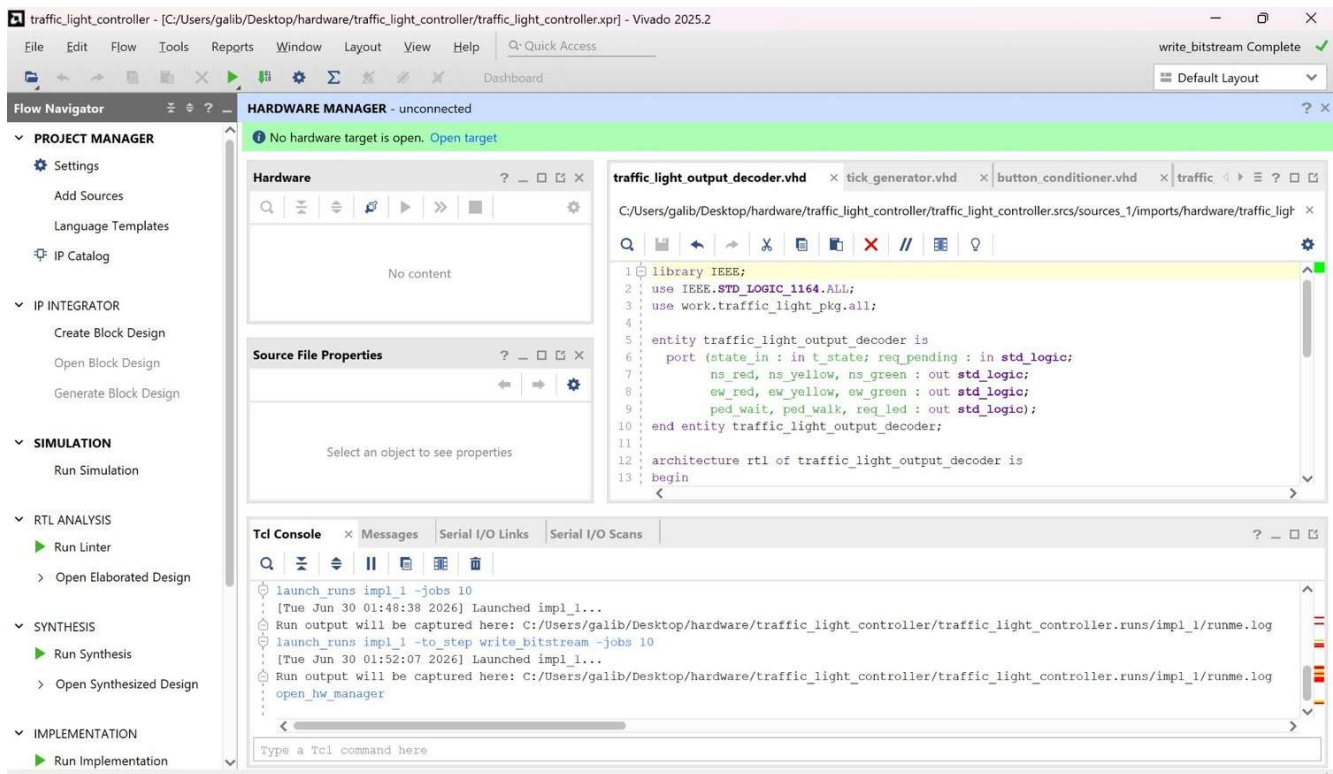


Figure 12: Vivado status showing write_bitstream Complete for the traffic_light_controller project.

8. PCB Design and Hardware Implementation

The PCB part represents the external interface for the traffic-light controller. It includes LED indicators, push buttons, a 12-pin header, voltage regulation and decoupling capacitors. The PCB package contains Altium design files, Gerber outputs, a PDF export, a BOM spreadsheet and a 3D view.

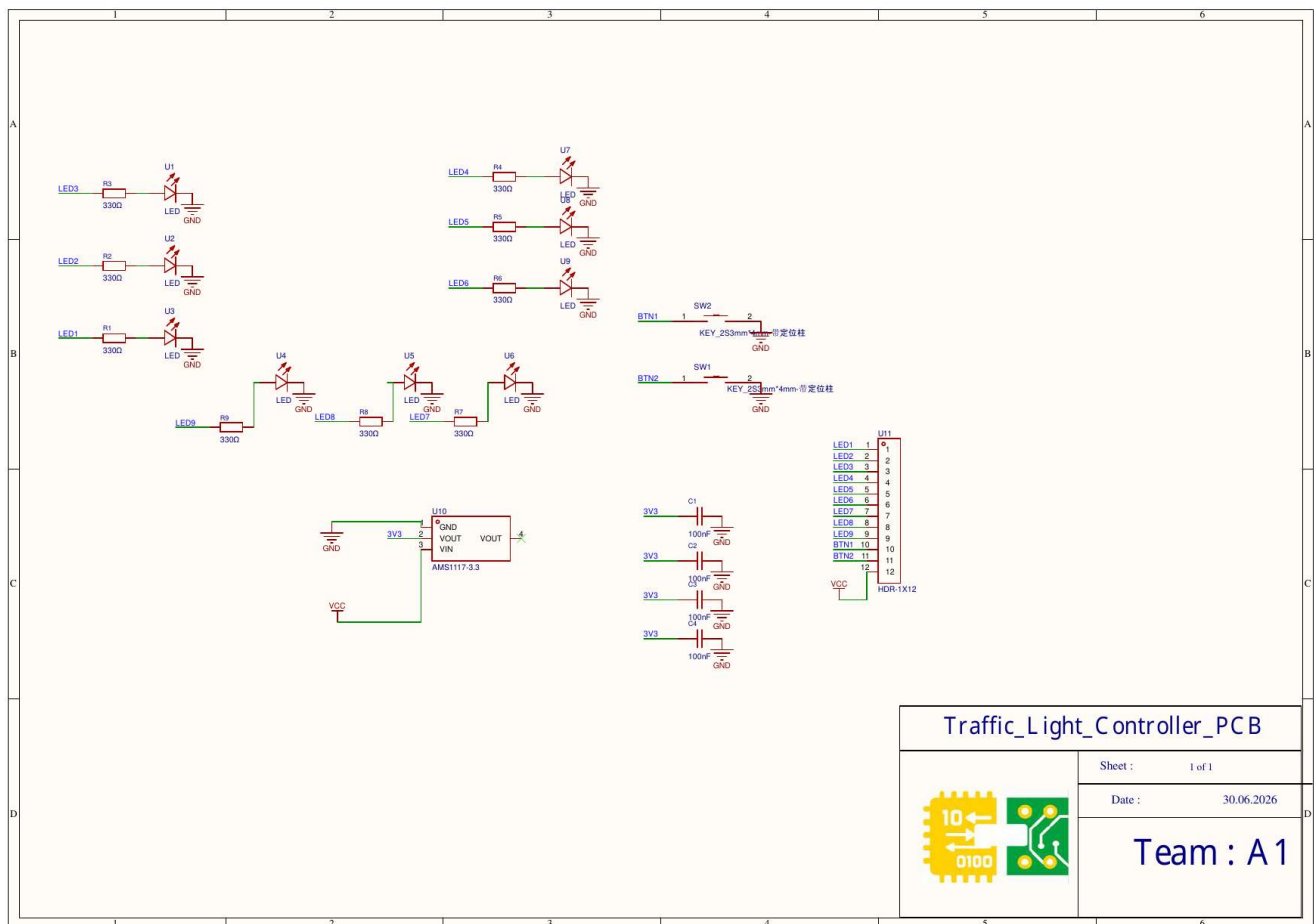


Figure 13: Schematic output showing LED channels, switches, 3.3 V regulator, decoupling capacitors and the 1x12 interface header.

8.1 Schematic Blocks

Block	Components	Purpose
LED indicators	U1-U9 and R1-R9, 330 ohm resistors	Represent NS red/yellow/green, EW red/yellow/green, pedestrian WAIT/WALK and request-status outputs.
Push-button inputs	SW1 and SW2	Provide reset and pedestrian-request inputs to the interface.
Board connector	U11, HDR-1X12	Carries LED, button, power and ground signals between the PCB and the FPGA/prototype interface.
Power regulation	U10, AMS1117-3.3	Provides a local 3.3 V supply rail.
Decoupling	C1-C4, 100 nF capacitors	Stabilizes the local supply and helps reduce switching noise.

8.2 PCB Layout and 3D View

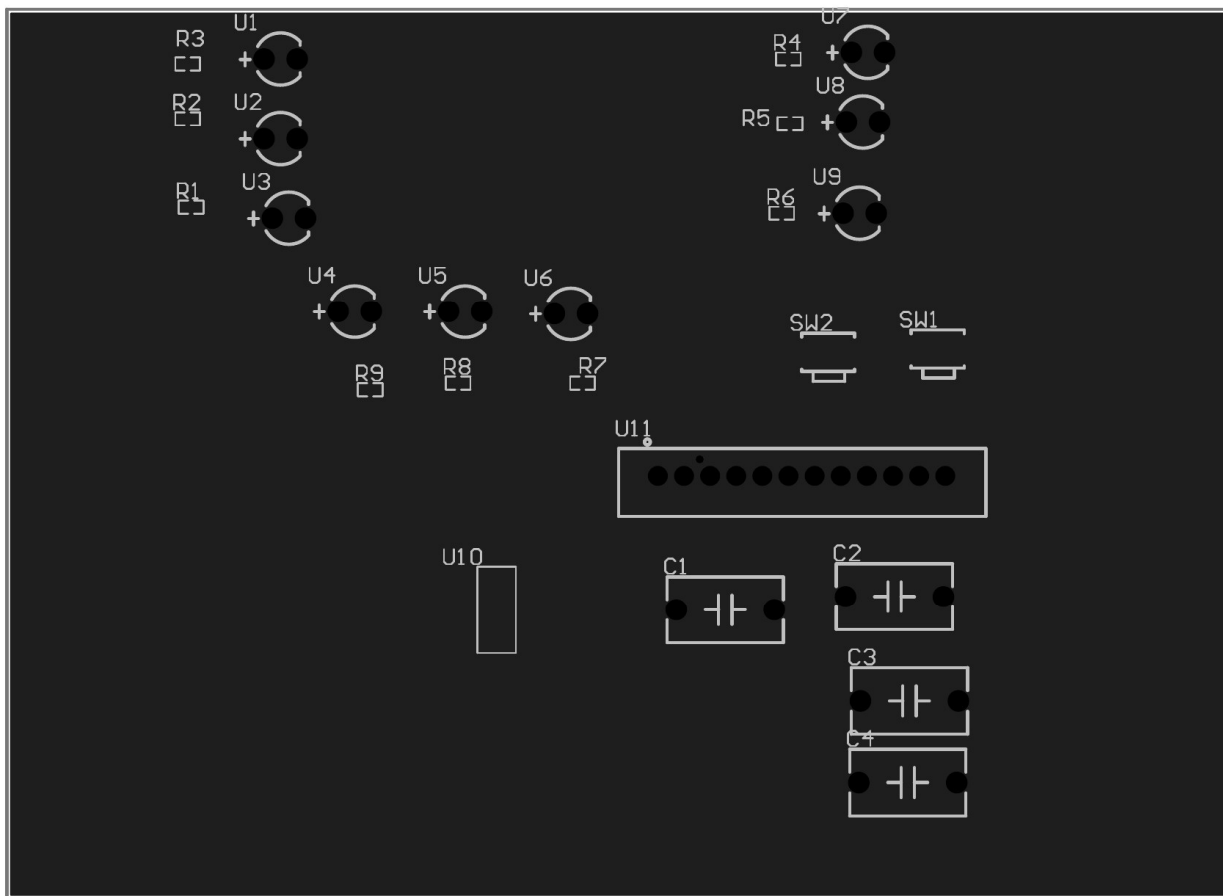


Figure 14: PCB layout/top-view output from the Altium project PDF.

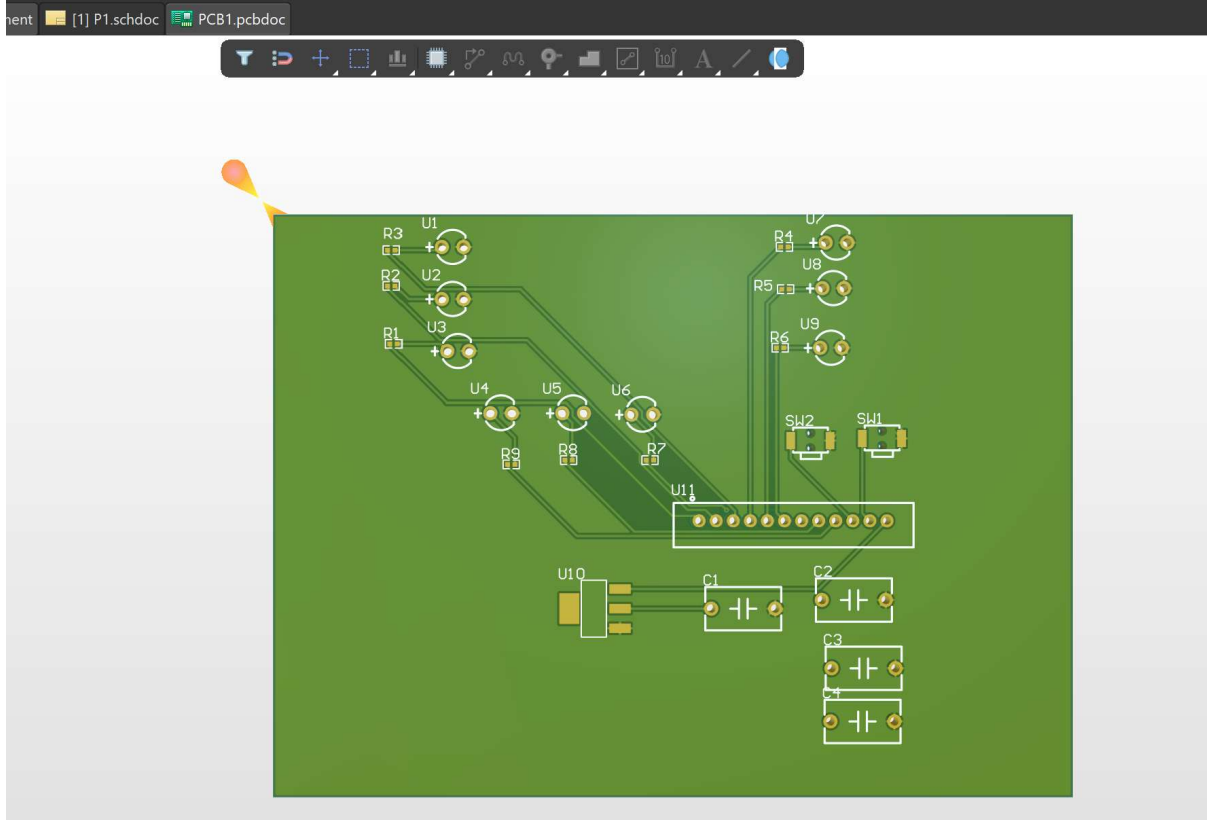


Figure 15: 3D PCB view showing the placement of LEDs, buttons, header and power components.

Board Stack Report

Figure 16: PCB output page showing layer-stack and board output information from the PDF export.

8.3 BOM Summary

Name	Description / role	Designators	Qty
100 nF capacitor	Supply decoupling capacitor with through-hole footprint.	C1, C2, C3, C4	4
330 ohm resistor	Current-limiting resistor for the LEDs.	R1-R9	9
Tactile switch	Push buttons for user, reset or pedestrian input functions.	SW1, SW2	2
LED	Indicators for traffic lights and pedestrian/request signals.	U1-U9	9
AMS1117-3.3	3.3 V linear voltage regulator.	U10	1
HDR-1X12	Interface header for external signal connection.	U11	1

8.4 Generated PCB Outputs

Output	Evidence / file	Purpose
Gerber files	Top/bottom copper, solder mask, top/bottom legend and profile Gerbers	Manufacturing data for PCB fabrication.
Status report	Status Report.Txt	Shows that 9 output files were generated at 21:09:13 on 30/06/2026.
PDF output	Traffic_Light_Controller.pdf	Contains schematic, PCB top view and PCB stack/report pages.
BOM spreadsheet	Traffic_Light_Controller.xlsx	Lists component descriptions, designators, footprints, library references and quantities.
3D view	PCB3D.png	Shows the mechanical placement and overall board appearance.

9. Discussion, Problems Faced and Design Decisions

1.Button conditioning: The pedestrian button was not connected directly to the FSM because a real button can bounce and is asynchronous to the clock. For this reason, synchronization, debouncing and edge detection were added.

2.Safe pedestrian handling: A pedestrian request is stored instead of interrupting the current vehicle green or yellow phase. The FSM serves the request only at S6_ALL_RED_2, where both vehicle directions are red.

3.Timing and simulation: The hardware design uses the 100 MHz board clock and a one-second tick for visible timing. During simulation, smaller timing values can be used to reduce waiting time while keeping the same design structure.

4.Timing constraints: The routed design meets the 100 MHz clock timing requirement. The timing report also indicates that input and output delays are not fully constrained, so these constraints should be added for a more complete production-style XDC file.

5.Hardware evidence: The available Vivado screenshot confirms successful bitstream generation. A photo or short video of the programmed board would be useful as additional evidence if the hardware demonstration is available.

6.PCB integration: The PCB can be used as an external LED and button interface. During hardware wiring, the 12-pin header order must match both the Nexys A7 pin assignments and the XDC constraints.

10. Conclusion and Future Improvements

The FPGA-based traffic-light controller with pedestrian request was completed as a modular VHDL design. It was simulated with a behavioral testbench and processed through Vivado synthesis, implementation, timing analysis, utilization reporting, power reporting and bitstream generation. The results show that the design is small, uses only a limited amount of Nexys A7-100T resources and meets the 100 MHz timing requirement.

The PCB part adds a project-specific external interface with LEDs, push buttons, a header, a regulator and decoupling capacitors. The generated PCB files include schematic and layout evidence, a 3D view, a BOM and Gerber manufacturing outputs. Overall, the project covers the complete lab workflow from concept and raw sketches to VHDL implementation, FPGA results and PCB documentation.

10.1 Future Improvements

- Add a photo or short video of the programmed board to support the bitstream-generation evidence.
- Add input and output delay constraints to make the XDC file more complete.
- Add a blinking pedestrian WAIT signal during S8_PED_CLEAR.
- Add a seven-segment countdown for the pedestrian WALK time.
- Add a night mode with flashing yellow outputs for the vehicle lights.
- Improve the PCB silkscreen by clearly separating the NS, EW and pedestrian LED groups.

Appendix A. VHDL Source-File Summary

This appendix shows short excerpts from the VHDL source files. The complete source files should be kept separate with the Vivado project.

traffic_light_pkg.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

package traffic_light_pkg is
    type t_state is (
        S0_RESET, S1_NS_GREEN, S2_NS_YELLOW, S3_ALL_RED_1,
        S4_EW_GREEN, S5_EW_YELLOW, S6_ALL_RED_2,
        S7_PED_WALK, S8_PED_CLEAR
    );
    constant C_NS_GREEN_TIME : integer := 5;
    constant C_EW_GREEN_TIME : integer := 5;
    constant C_YELLOW_TIME   : integer := 2;
    constant C_ALLRED_TIME   : integer := 1;
    constant C_WALK_TIME     : integer := 4;
    constant C_PEDCLEAR_TIME : integer := 2;
    constant C_STARTUP_TIME  : integer := 1;
end package traffic_light_pkg;
```

tick_generator.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity tick_generator is
    generic (G_CLK_FREQ_HZ : integer := 100_000_000);
    port (clk : in std_logic; rst : in std_logic; tick_1s : out std_logic);
end entity tick_generator;
```

architecture rtl of tick_generator is

```
    signal r_counter : integer range 0 to 100_000_000 - 1 := 0;
begin
    process(clk)
    begin
        if rising_edge(clk) then
            if rst = '1' then
                r_counter <= 0; tick_1s <= '0';
            elsif r_counter = G_CLK_FREQ_HZ - 1 then
                r_counter <= 0; tick_1s <= '1';
            else
                r_counter <= r_counter + 1; tick_1s <= '0';
            end if;
        end if;
    end process;
end architecture rtl;
```

button_conditioner.vhd

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity button_conditioner is
    generic (G_DEBOUNCE_TICKS : integer := 20);
    port (clk : in std_logic; rst : in std_logic;
          btn_raw : in std_logic; btn_pulse : out std_logic);
end entity button_conditioner;

architecture rtl of button_conditioner is
    signal r_sync0, r_sync1 : std_logic := '0';
    signal r_debounce_cnt : integer range 0 to 20 := 0;
    signal r_stable : std_logic := '0';
    signal r_prev_stable : std_logic := '0';
```

```
begin
  process(clk)
  begin
    if rising_edge(clk) then
      if rst = '1' then
        r_sync0 <= '0'; r_sync1 <= '0';
        r_debounce_cnt <= 0; r_stable <= '0';
        r_prev_stable <= '0'; btn_pulse <= '0';
      else
        r_sync0 <= btn_raw; r_sync1 <= r_sync0;
        if r_sync1 = r_stable then
          r_debounce_cnt <= 0;
        else
          if r_debounce_cnt = G_DEBOUNCE_TICKS then
            r_stable <= r_sync1; r_debounce_cnt <= 0;
          else
            r_debounce_cnt <= r_debounce_cnt + 1;
          end if;
        end if;
        r_prev_stable <= r_stable;
        if r_stable = '1' and r_prev_stable = '0' then
          btn_pulse <= '1';
        else
          btn_pulse <= '0';
        end if;
      end if;
    end if;
  end process;
end architecture rtl;
```

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.traffic_light_pkg.all;

entity traffic_light_fsm is
    port (clk : in std_logic; rst : in std_logic;
          tick_1s : in std_logic; ped_btn_pulse : in std_logic;
          state_out : out t_state; request_pending : out std_logic);
end entity traffic_light_fsm;

architecture rtl of traffic_light_fsm is
    signal r_state : t_state := S0_RESET;
    signal r_timer : integer := 0;
    signal r_req_pending : std_logic := '0';
begin
    state_out <= r_state;
    request_pending <= r_req_pending;

    process(clk)
    begin
        ...
        if tick_1s = '1' then
            r_timer <= r_timer + 1;
            case r_state is
                when S0_RESET =>
                    if r_timer >= C_STARTUP_TIME-1 then r_state <= S1_NS_GREEN; r_timer <= 0; end if;
                when S1_NS_GREEN =>
                    if r_timer >= C_NS_GREEN_TIME-1 then r_state <= S2_NS_YELLOW; r_timer <= 0; end
            if;
                when S2_NS_YELLOW =>
                    if r_timer >= C_YELLOW_TIME-1 then r_state <= S3_ALL_RED_1; r_timer <= 0; end if;
                when S3_ALL_RED_1 =>
                    if r_timer >= C_ALLRED_TIME-1 then r_state <= S4_EW_GREEN; r_timer <= 0; end if;
                when S4_EW_GREEN =>
                    if r_timer >= C_EW_GREEN_TIME-1 then r_state <= S5_EW_YELLOW; r_timer <= 0;

```

```

end if;

when S5_EW_YELLOW =>
    if r_timer >= C_YELLOW_TIME-1 then r_state <= S6_ALL_RED_2; r_timer <= 0; end if;
when S6_ALL_RED_2 =>
    if r_timer >= C_ALLRED_TIME-1 then
        r_timer <= 0;
        if r_req_pending = '1' then r_state <= S7_PED_WALK;
        else r_state <= S1_NS_GREEN; end if;
    end if;
when S7_PED_WALK =>
    r_req_pending <= '0';
    if r_timer >= C_WALK_TIME-1 then r_state <= S8_PED_CLEAR; r_timer <= 0; end if;
when S8_PED_CLEAR =>
    if r_timer >= C_PEDCLEAR_TIME-1 then r_state <= S1_NS_GREEN; r_timer <= 0; end if;
    when others => r_state <= S0_RESET; r_timer <= 0;
end case;
end if;
end if;

```

traffic_light_output_decoder.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.traffic_light_pkg.all;

entity traffic_light_output_decoder is
    port (state_in : in t_state; req_pending : in std_logic;
          ns_red, ns_yellow, ns_green : out std_logic;
          ew_red, ew_yellow, ew_green : out std_logic;
          ped_wait, ped_walk, req_led : out std_logic);
end entity traffic_light_output_decoder;

architecture rtl of traffic_light_output_decoder is
begin

```

```

req_led <= req_pending;
process(state_in)
begin
    ns_red<='1'; ns_yellow<='0'; ns_green<='0';
    ew_red<='1'; ew_yellow<='0'; ew_green<='0';
    ped_wait<='1'; ped_walk<='0';
    case state_in is
        when S1_NS_GREEN => ns_red<='0'; ns_green<='1';
        when S2_NS_YELLOW => ns_red<='0'; ns_yellow<='1';
        when S4_EW_GREEN => ew_red<='0'; ew_green<='1';
        when S5_EW_YELLOW => ew_red<='0'; ew_yellow<='1';
        when S7_PED_WALK => ped_wait<='0'; ped_walk<='1';
        when others => null;
    end case;
end process;
end architecture rtl;

```

traffic_light_top.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.traffic_light_pkg.all;

entity traffic_light_top is
    port (clk, rst, btn_ped : in std_logic;
          ns_red, ns_yellow, ns_green : out std_logic;
          ew_red, ew_yellow, ew_green : out std_logic;
          ped_wait, ped_walk, req_led : out std_logic);
end entity traffic_light_top;

architecture structural of traffic_light_top is
    signal s_tick_1s, s_btn_pulse, s_req_pending : std_logic;
    signal s_state : t_state;
begin

```

```

u_tick : entity work.tick_generator
  generic map(G_CLK_FREQ_HZ => 100_000_000)
  port map(clk=>clk, rst=>rst, tick_1s=>s_tick_1s);
u_btn : entity work.button_conditioner
  generic map(G_DEBOUNCE_TICKS => 20)
  port map(clk=>clk, rst=>rst, btn_raw=>btn_ped, btn_pulse=>s_btn_pulse);
u_fsm : entity work.traffic_light_fsm
  port map(clk=>clk, rst=>rst, tick_1s=>s_tick_1s,
    ped_btn_pulse=>s_btn_pulse, state_out=>s_state,
    request_pending=>s_req_pending);
u_dec : entity work.traffic_light_output_decoder
  port map(state_in=>s_state, req_pending=>s_req_pending,
    ns_red=>ns_red, ns_yellow=>ns_yellow, ns_green=>ns_green,
    ew_red=>ew_red, ew_yellow=>ew_yellow, ew_green=>ew_green,
    ped_wait=>ped_wait, ped_walk=>ped_walk, req_led=>req_led);
end architecture structural;

```

traffic_light_fsm_tb.vhd

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use work.traffic_light_pkg.all;

entity traffic_light_fsm_tb is
end entity traffic_light_fsm_tb;

architecture sim of traffic_light_fsm_tb is
  constant C_CLK_PERIOD : time := 10 ns;
  signal tb_clk, tb_rst, tb_btn_ped : std_logic := '0';
  signal tb_ns_red, tb_ns_yel, tb_ns_grn : std_logic;
  signal tb_ew_red, tb_ew_yel, tb_ew_grn : std_logic;
  signal tb_ped_wait, tb_ped_walk, tb_req_led : std_logic;
begin
  tb_clk <= not tb_clk after C_CLK_PERIOD/2;

```



```

UUT : entity work.traffic_light_top
  port map(clk=>tb_clk, rst=>tb_rst, btn_ped=>tb_btn_ped,
           ns_red=>tb_ns_red, ns_yellow=>tb_ns_yel, ns_green=>tb_ns_grn,
           ew_red=>tb_ew_red, ew_yellow=>tb_ew_yel, ew_green=>tb_ew_grn,
           ped_wait=>tb_ped_wait, ped_walk=>tb_ped_walk, req_led=>tb_req_led);
stim_proc : process
begin
...
  wait for 150 us;
  report "TB-04 to TB-08: Check waveforms.";
  wait;
end process;

process(tb_clk)
begin
  if rising_edge(tb_clk) then
    assert NOT(tb_ns_grn='1' AND tb_ew_grn='1')
      report "SAFETY FAIL: NS+EW green together!" severity FAILURE;
    assert NOT(tb_ped_walk='1' AND (tb_ns_grn='1' OR tb_ew_grn='1'))
      report "SAFETY FAIL: PED_WALK with vehicle green!" severity FAILURE;
  end if;
end process;
end architecture sim;

```

Appendix B. Nexys A7 Pin Mapping and XDC Constraints

The XDC block below follows the top-level diagram and the Nexys A7 LED/button mapping. Before programming the hardware, it should be compared with the official Nexys A7 master XDC and the final Vivado constraints file.

```

## Clock
create_clock -period 10.000 -name clk -waveform {0.000 5.000} [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]

## Push buttons

```

```

set_property PACKAGE_PIN M18 [get_ports btn_ped] # BTNL pedestrian request
set_property PACKAGE_PIN N17 [get_ports rst]    # BTNC reset
set_property IOSTANDARD LVCMOS33 [get_ports {btn_ped rst}]

## LEDs
set_property PACKAGE_PIN H17 [get_ports ns_red]    # LD0
set_property PACKAGE_PIN K15 [get_ports ns_yellow] # LD1
set_property PACKAGE_PIN J13 [get_ports ns_green]  # LD2
set_property PACKAGE_PIN N14 [get_ports ew_red]    # LD3
set_property PACKAGE_PIN R18 [get_ports ew_yellow] # LD4
set_property PACKAGE_PIN V17 [get_ports ew_green]  # LD5
set_property PACKAGE_PIN U17 [get_ports ped_wait]  # LD6
set_property PACKAGE_PIN U16 [get_ports ped_walk]  # LD7
set_property PACKAGE_PIN V16 [get_ports req_led]   # LD8
set_property IOSTANDARD LVCMOS33 [get_ports {ns_red ns_yellow ns_green ew_red ew_yellow
ew_green ped_wait ped_walk req_led}]

```

Appendix C. BOM Summary

The BOM summary below was taken from the Traffic_Light_Controller.xlsx file used for the PCB package.

Name	Full designators	Footprint	LibRef	Qty
100nF	C1, C2, C3, C4	CAP-TH_L9.0-W5.0-P7.50-D0.6-1	CFC2G104JD11J5085100	4
330 ohm	R1, R2, R3, R4, R5, R6, R7, R8, R9	R0402	0402WGF3300TCE	9
KEY_2S3mm*4mm	SW1, SW2	KEY-SMD_L4.0-W3.7-LS4.8	KEY_2S3mm*4mm	2
LED	U1, U2, U3, U4, U5, U6, U7, U8, U9	LED-TH_BD4.0-P2.54-FD_RED	LED	9
AMS1117-3.3	U10	SOT-223-3_L6.5-W3.4-P2.30-LS7.0-BR	AMS1117-3.3_C6186	1

HDR-1X12	U11	CONN-TH_HDR- 1X12	HDR-1X12	1
----------	-----	----------------------	----------	---